

Curso desarrollo de aplicaciones con tecnologías web
Módulo formativo 2: Programación web en el entorno servidor

Unidad formativa 1: Desarrollo de aplicaciones web en el entorno servidor

Lección 4: Lenguajes de Programación de Aplicaciones Web en el Lado Servidor

Características de los Lenguajes de Programación Web en Servidor

Tipos y Características de los Lenguajes de Uso Común

Plataformas para el Desarrollo en las Capas Servidor

Criterios en la Elección de un Lenguaje de Programación Web en Servidor

Características Generales

Gestión de la Configuración

Gestión de la Seguridad

Transacciones y Persistencia

Características de los Lenguajes de Programación Web en Servidor

Los lenguajes de servidor procesan peticiones, generan respuestas dinámicas y gestionan bases de datos.

Tipos y Características de los Lenguajes de Uso Común

- **Interpretados Orientados a Servidor**

Lenguajes como PHP y Python ejecutan código sin necesidad de compilación previa.

- **Lenguajes de Cliente Interpretados en Servidor**

JavaScript con Node.js permite ejecutar código del lado servidor utilizando V8.

- **Lenguajes Compilados**

Java y C# requieren compilación, proporcionando mayor eficiencia en tiempo de ejecución.

Criterios en la Elección de un Lenguaje de Programación Web en Servidor

La elección depende de la facilidad de uso, escalabilidad, seguridad y compatibilidad con bases de datos.

Características Generales

Tipos de Datos

PHP maneja tipos como enteros, flotantes, cadenas y arrays.

```
$numero = 10;  
$texto = "Hola";  
$lista = ["manzana", "pera", "uva"];
```

Clases

Ejemplo de una clase en PHP:

```
class Persona {  
    public $nombre;  
    public function __construct($nombre) {  
        $this->nombre = $nombre;  
    }  
}
```

Operadores Básicos y Manipulación de Cadenas

```
$nombre = "Juan";  
echo "Hola " . $nombre;
```

Estructuras de Control (Bucles y Condicionales)

```
if ($numero > 5) {  
    echo "Número mayor que 5";  
}
```

Módulos o Paquetes

Los módulos permiten organizar mejor el código y reutilizar funcionalidades.

```
include 'utilidades.php';
```

Herencia

Ejemplo en PHP:

```
class Estudiante extends Persona {  
    public $curso;  
}
```

Gestión de Bibliotecas

PHP permite el uso de bibliotecas como Composer.

Gestión de la Configuración

Configuración de archivos en PHP:

```
$datos = parse_ini_file("config.ini");
```

Gestión de la Seguridad

Identificación, Autenticación y Autorización

```
session_start();  
$_SESSION['usuario'] = "admin";
```

Gestión de Sesiones

```
session_destroy();
```

Gestión de Errores

- Técnicas de Recuperación de Errores

```
set_error_handler("miManejadorErrores");
```

- Programación de Excepciones

```
try {  
    throw new Exception("Error encontrado");  
} catch (Exception $e) {  
    echo $e->getMessage();  
}
```

Transacciones y Persistencia

Acceso a Bases de Datos y Conectores

Ejemplo con MySQL y PDO:

```
$pdo = new PDO('mysql:host=localhost;dbname=prueba', 'usuario', 'clave');
```

Componentes en Servidor. Ventajas e inconvenientes en el uso de contenedores de componentes.

Los componentes en servidor permiten modularidad y reutilización de código.

Modelos de Desarrollo - MVC

MVC separa la aplicación en Modelo (datos), Vista (interfaz) y Controlador (lógica).

- **Modelo: Programación de Acceso a Datos**

Ejemplo de una clase modelo:

```
class Usuario
{
    public function obtenerTodos()
    {
        return "Consulta de usuarios";
    }
}
```


- **Vista: Desarrollo de Aplicaciones en Cliente**

Ejemplo de una vista en PHP:

```
<h1>Bienvenido <php echo $usuario; ?></h1>
```

- **Programación del Controlador**

Ejemplo de un controlador en PHP:

```
class ControladorUsuario
{
    public function mostrar()
    {
        $usuario = new Usuario();
        echo $usuario->obtenerTodos();
    }
}
```

Documentación del Software

Ejemplo de documentación en PHP con PHPDoc:

```
/**
 * Clase de usuario
 * @author Nombre
 */
class Usuario {}
```